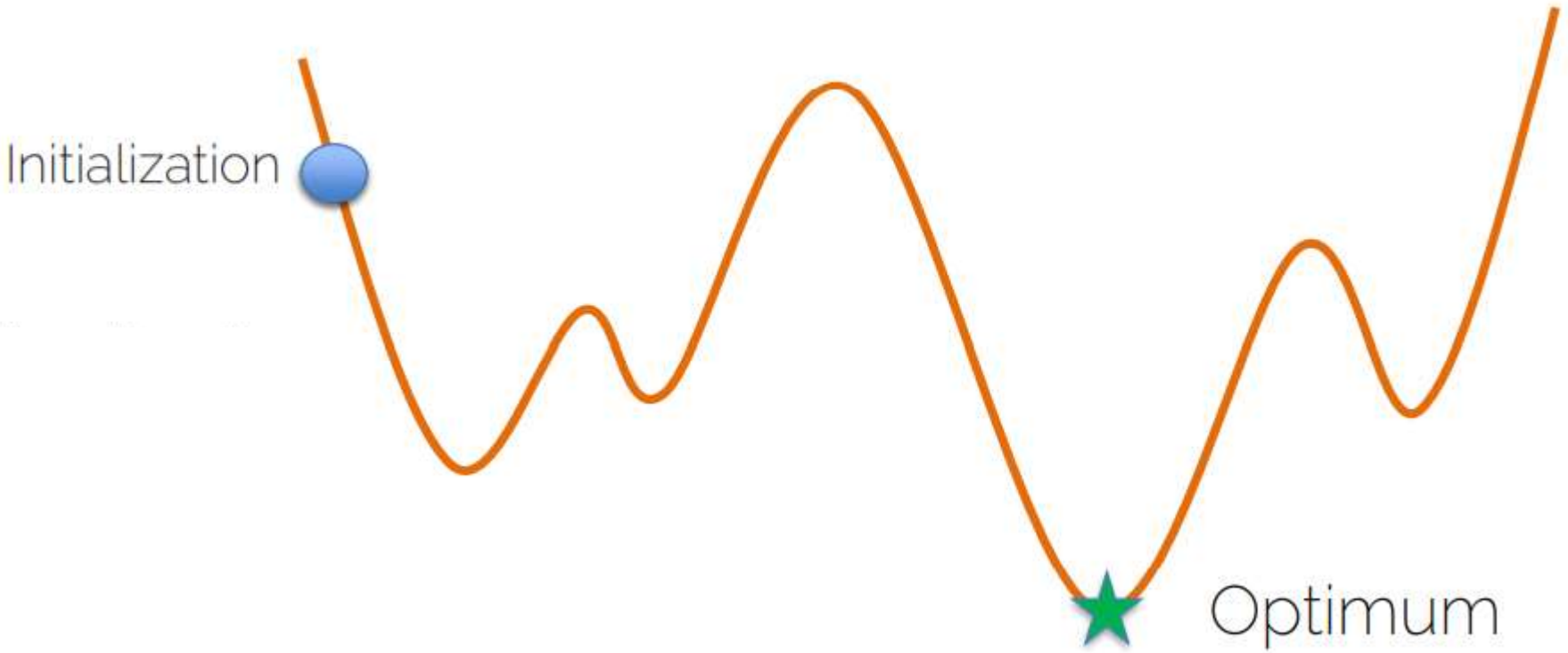
Optimization

Gradient Descent

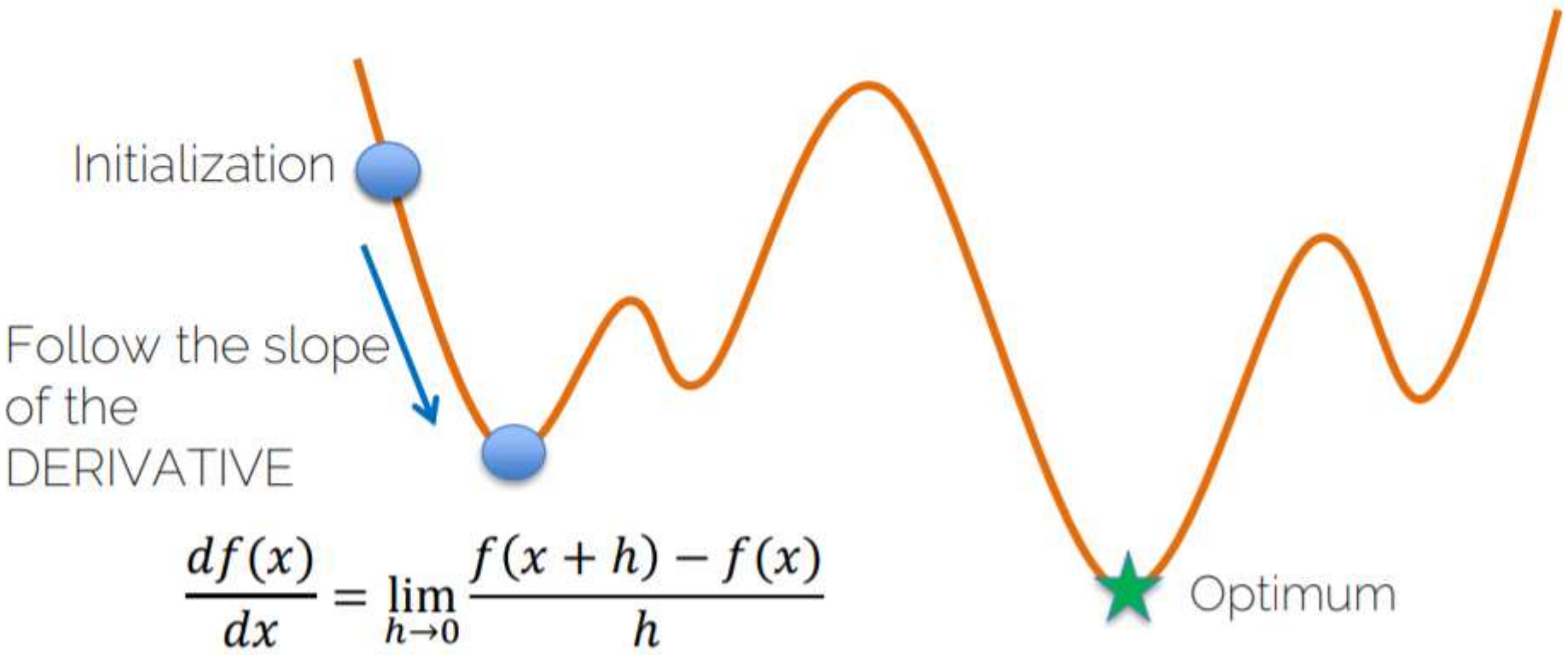
$$x^* = \arg\min f(x)$$



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent

$$x^* = \arg\min f(x)$$



$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent

- From derivative to gradient

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \longrightarrow \nabla_x f(x)$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent

- From derivative to gradient

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \longrightarrow \nabla_x f(x)$$

Direction of greatest increase of the function

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

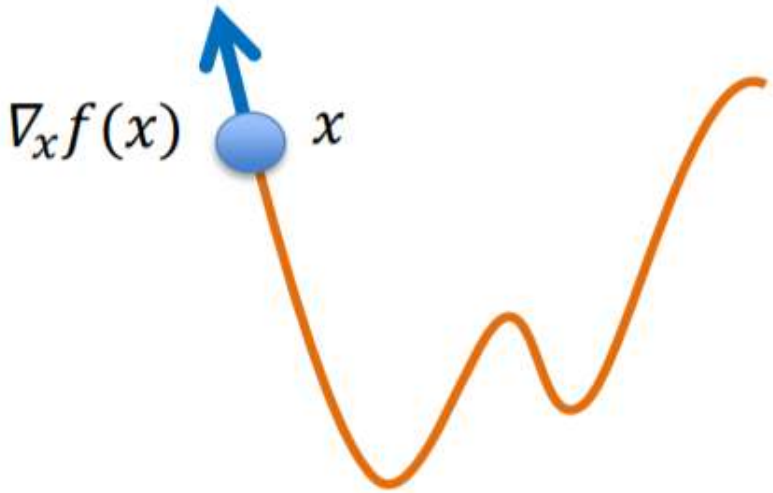
Gradient Descent

- From derivative to gradient

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \longrightarrow \nabla_x f(x)$$

Direction of greatest increase of the function

- Gradient steps in direction of negative gradient



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

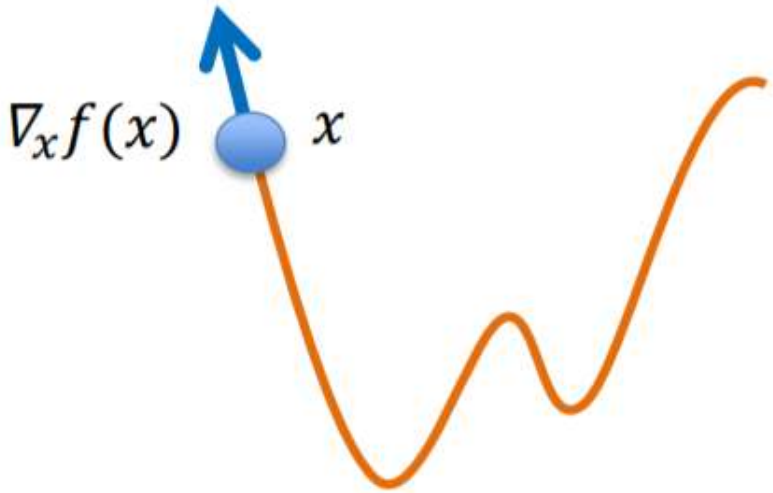
Gradient Descent

- From derivative to gradient

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \longrightarrow \nabla_x f(x)$$

Direction of greatest increase of the function

- Gradient steps in direction of negative gradient



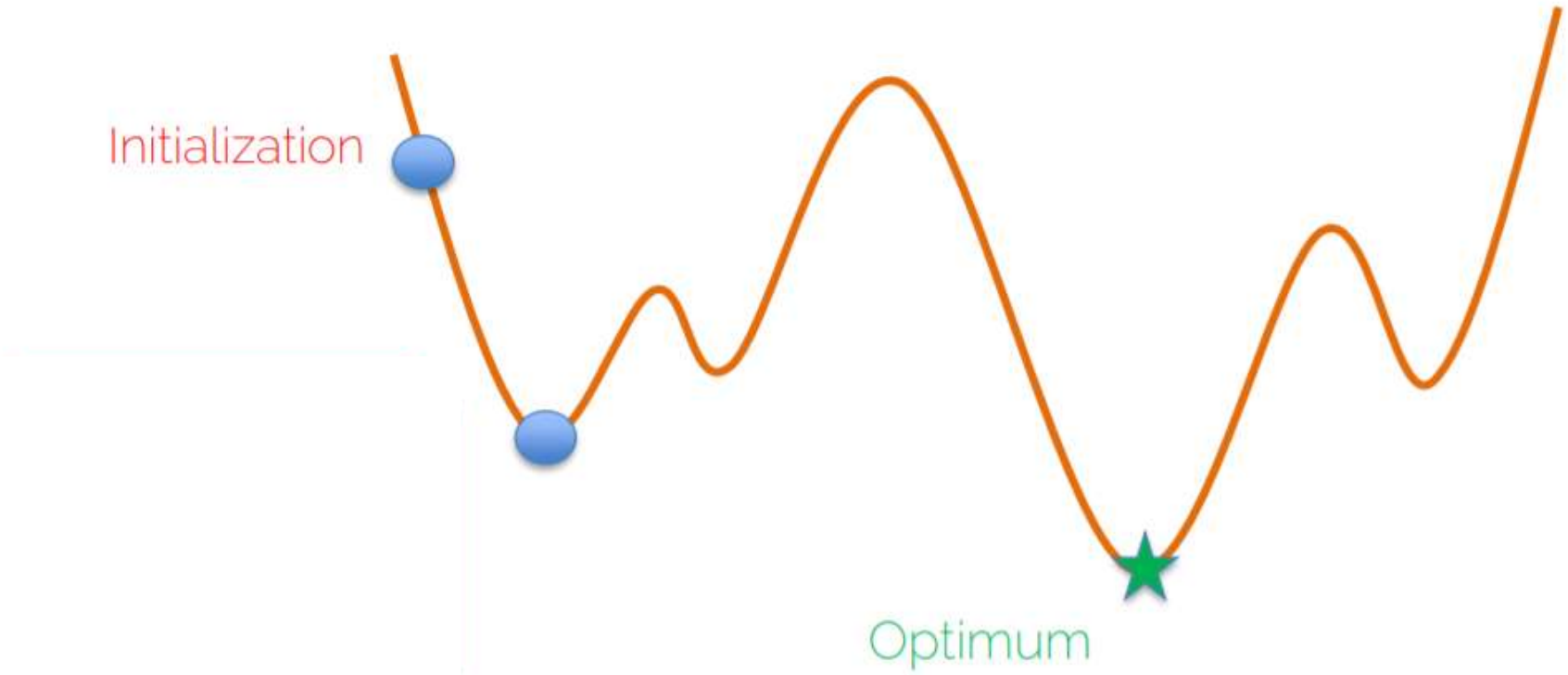
$$x' = x - \alpha \nabla_x f(x)$$

Learning rate

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent

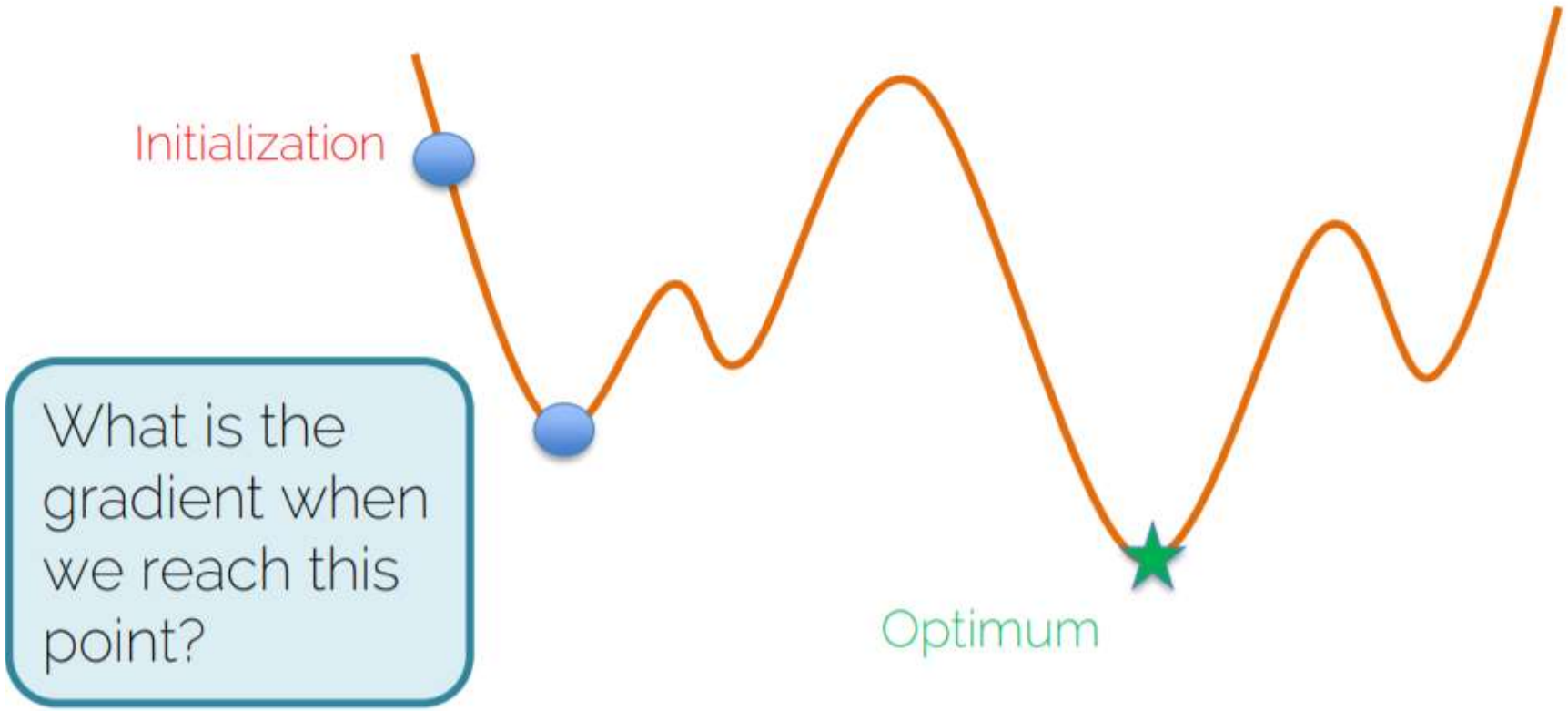
$$\boldsymbol{x}^* = \arg\min f(\boldsymbol{x})$$



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent

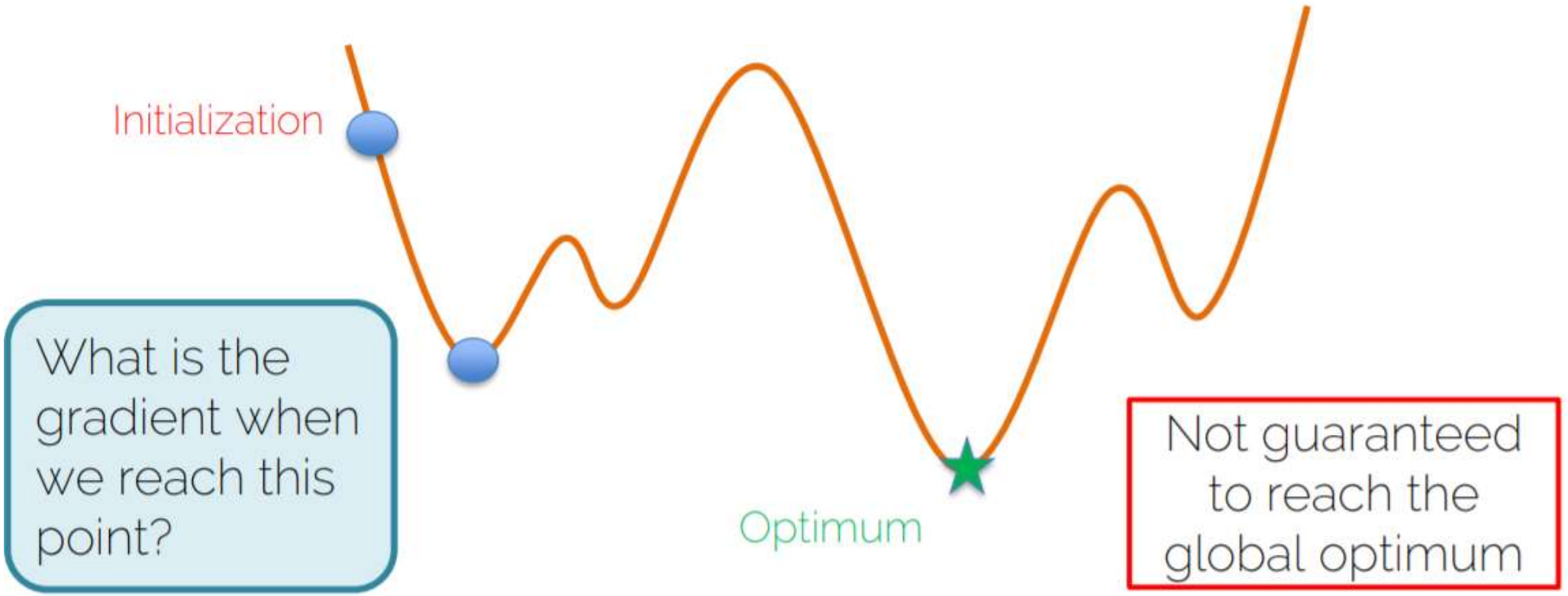
$$\boldsymbol{x}^* = \arg\min f(\boldsymbol{x})$$



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent

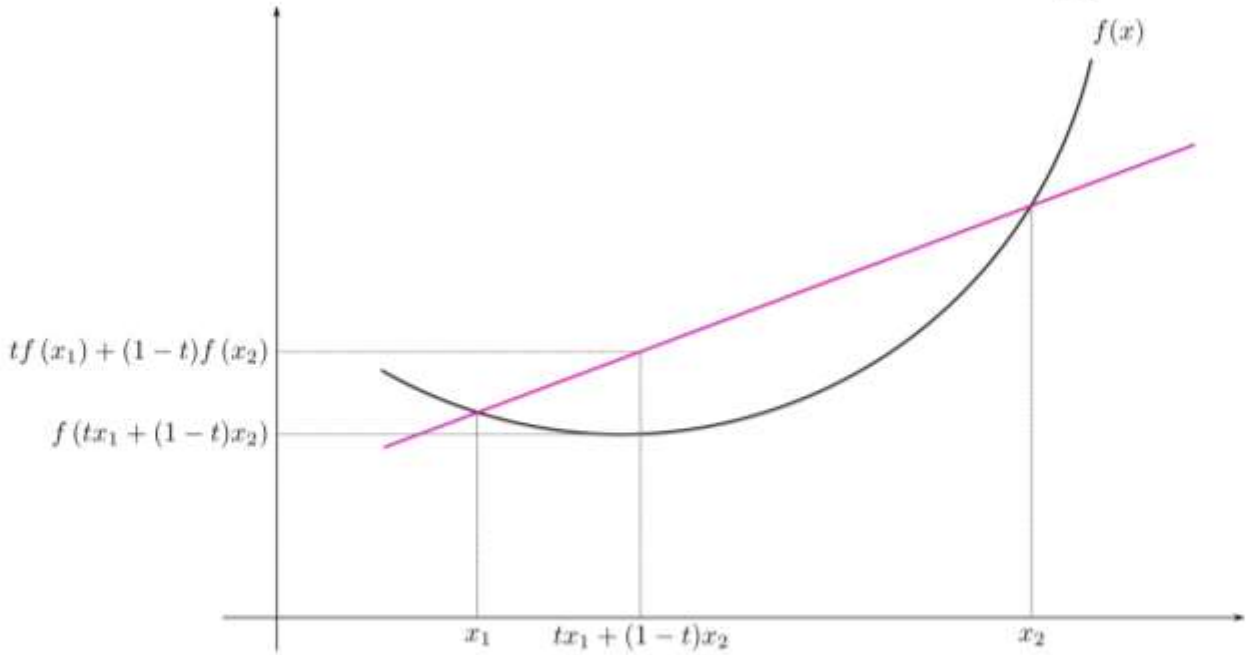
$$x^* = \arg\min f(x)$$



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convergence of Gradient Descent

Convex function: all local minima are global minima



Source: https://en.wikipedia.org/wiki/Convex_function#/media/File:ConvexFunction.svg

If line/plane segment between any two points lies above or on the graph

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convergence of Gradient Descent

Neural networks are non-convex

- many (different) local minima
- no (practical) way to say which is globally optimal

Convergence of Gradient Descent

$$x' = x - \alpha \nabla_x f(x)$$

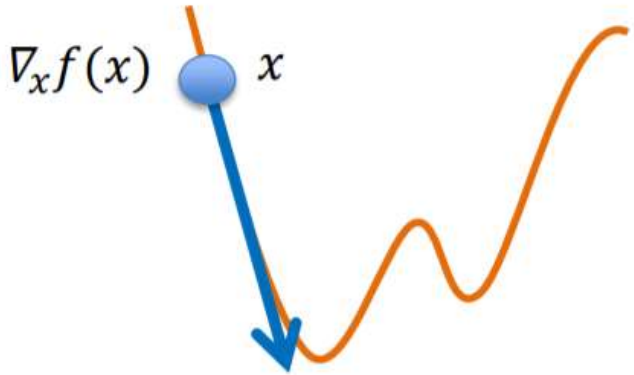
Learning rate

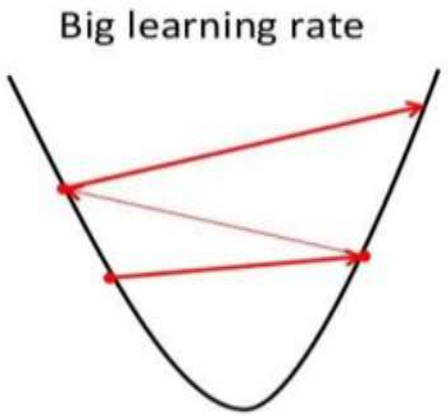
Source: https://builtin.com/data-science/gradient-descent

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convergence of Gradient Descent

Big
learning rate





$$x' = x - \alpha \nabla_x f(x)$$

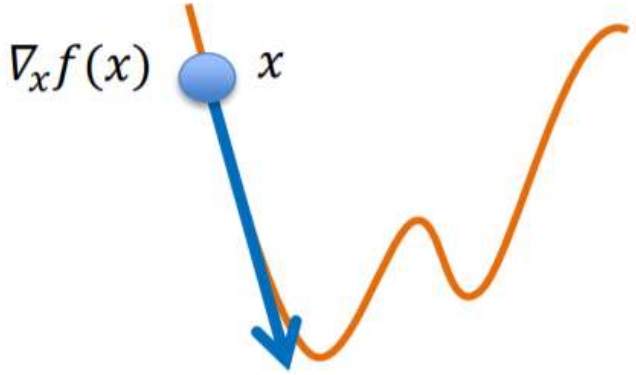
Learning rate

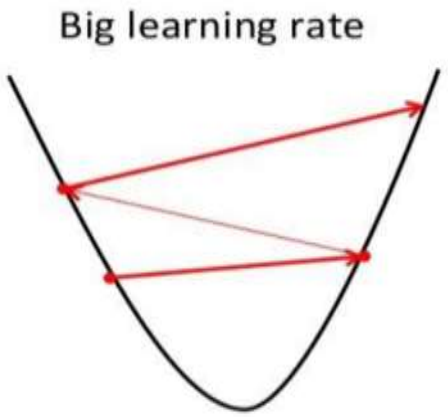
Source: https://builtin.com/data-science/gradient-descent

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Convergence of Gradient Descent

Big
learning rate

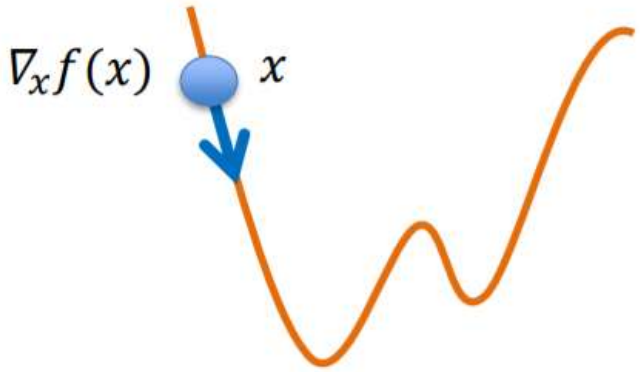


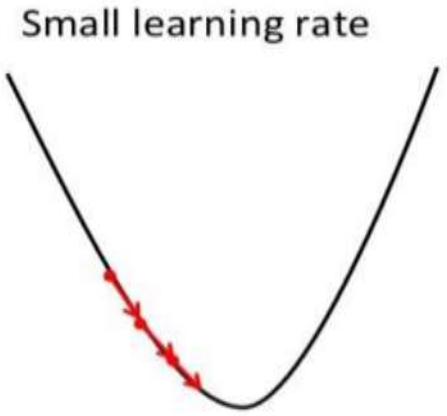


$$x' = x - \alpha \nabla_x f(x)$$

Learning rate

Small
learning rate

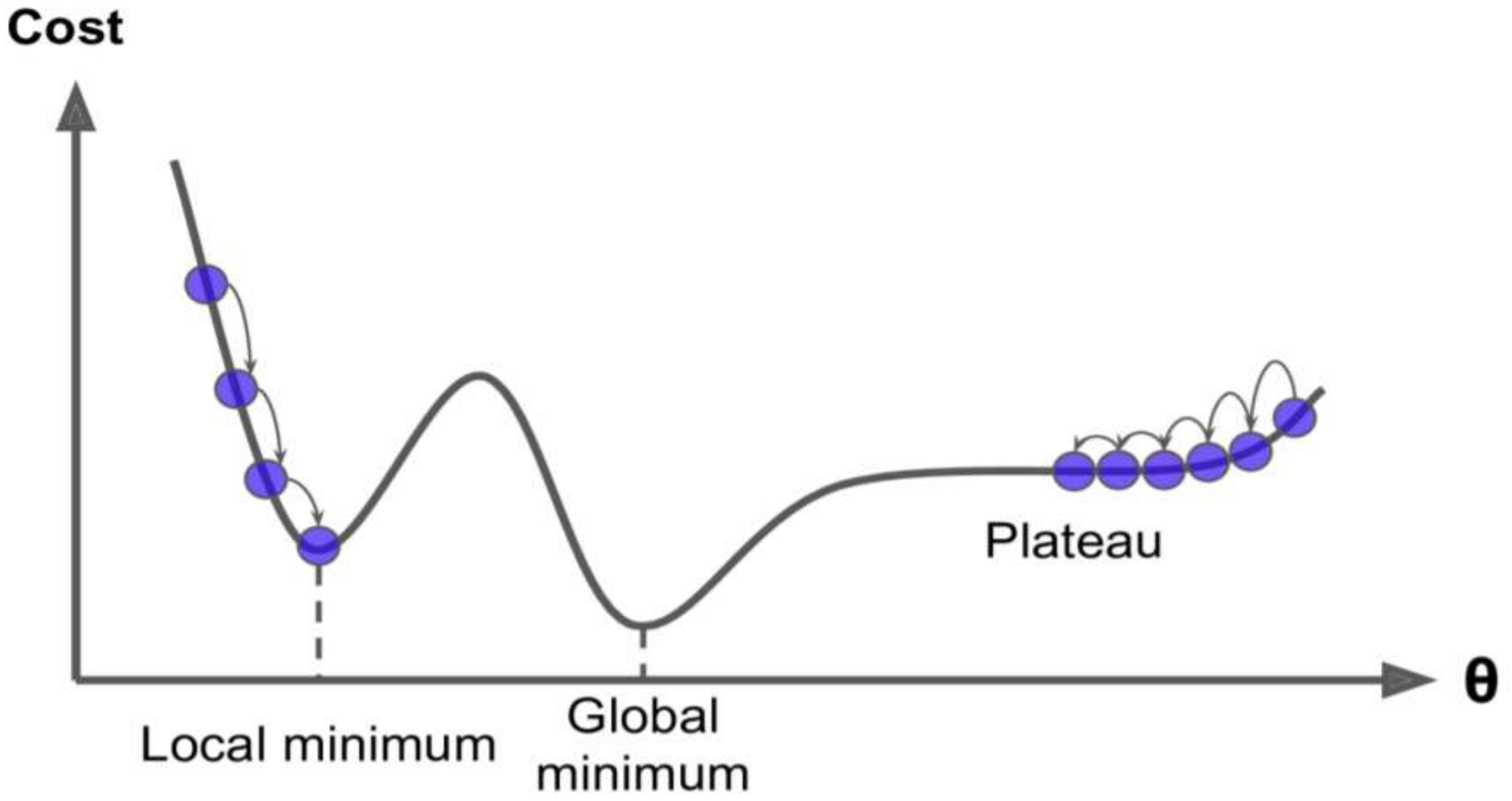




Source: https://builtin.com/data-science/gradient-descent

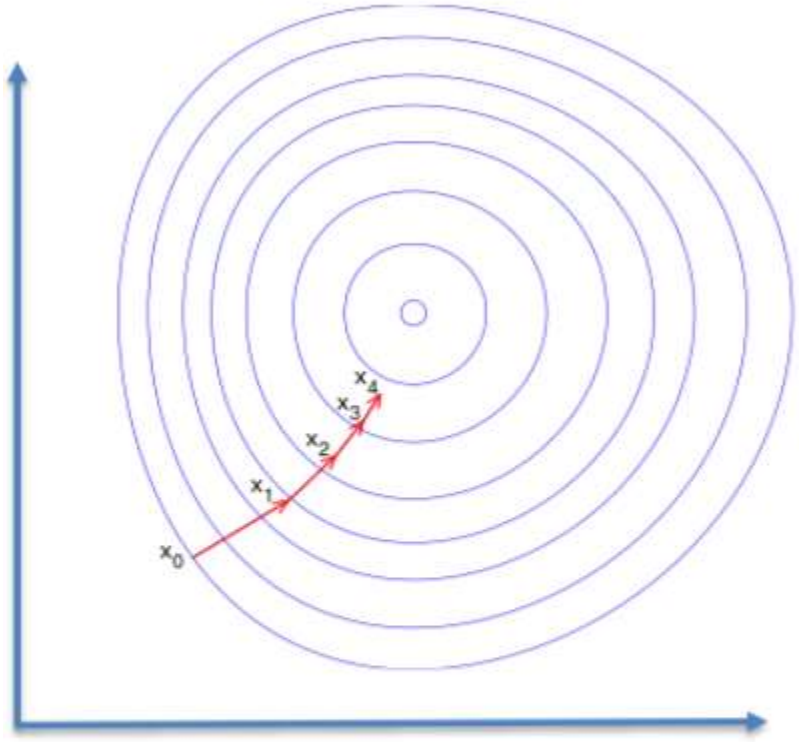
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

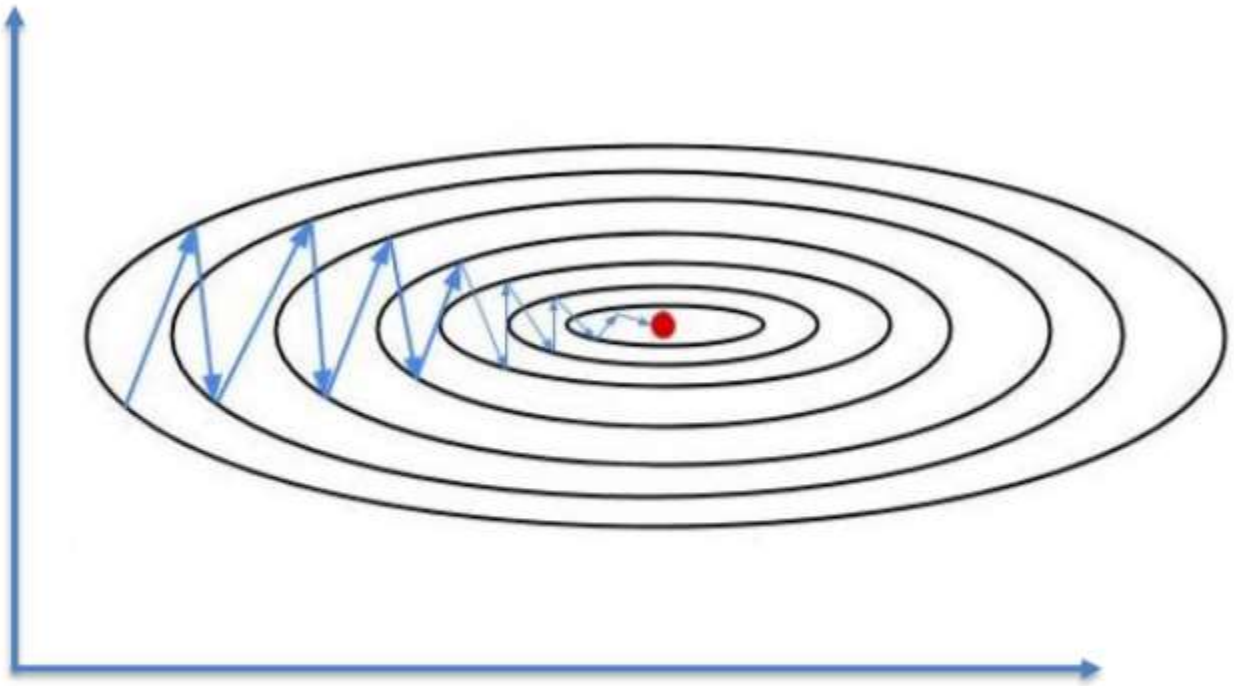
Convergence of Gradient Descent



Source: A. Geron

Gradient Descent: Multiple Dimensions

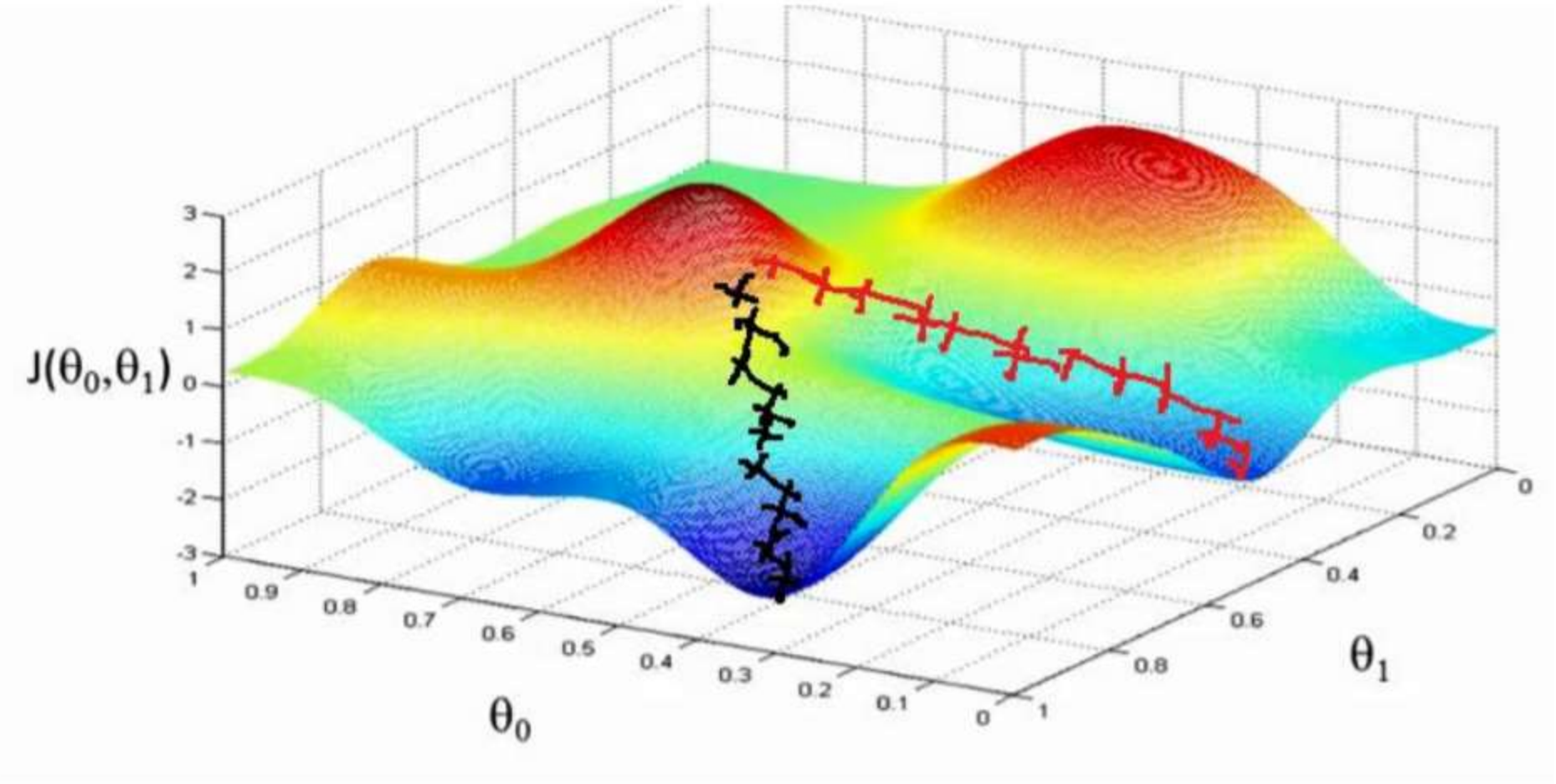




Source: builtin.com/data-science/gradient-descent

Various ways to visualize…

Gradient Descent: Multiple Dimensions



Source: http://blog.datumbox.com/wp-content/uploads/2013/10/gradient-descent.png

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample

- Given a loss function L and a single training sample $\{x_i, y_i\}$

Gradient Descent: Single Training Sample

- Given a loss function $\boldsymbol{L}$ and a single training sample $\{\boldsymbol{x}_i, \boldsymbol{y}_i\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample

- Given a loss function $\boldsymbol{L}$ and a single training sample $\{\boldsymbol{x}_i, \boldsymbol{y}_i\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$
- Cost $L_i(\boldsymbol{\theta}, \boldsymbol{x}_i, \boldsymbol{y}_i)$
  - $\boldsymbol{\theta} = \arg\min L_i(\boldsymbol{x}_i, \boldsymbol{y}_i)$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample

- Given a loss function $\boldsymbol{L}$ and a single training sample $\{\boldsymbol{x}_i, \boldsymbol{y}_i\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$
- Cost $\boldsymbol{L}_i(\boldsymbol{\theta}, \boldsymbol{x}_i, \boldsymbol{y}_i)$
  - $\boldsymbol{\theta} = \arg\min L_i(\boldsymbol{x}_i, \boldsymbol{y}_i)$
- Gradient Descent:

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample

- Given a loss function $\boldsymbol{L}$ and a single training sample $\{\boldsymbol{x_i}, \boldsymbol{y_i}\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$
- Cost $\boldsymbol{L_i(\theta, x_i, y_i)}$
  - $\boldsymbol{\theta} = \arg\min L_i(\boldsymbol{x_i}, \boldsymbol{y_i})$
- Gradient Descent:
  - Initialize $\boldsymbol{\theta^1}$ with 'random' values (more to that later)

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample

- Given a loss function $\boldsymbol{L}$ and a single training sample $\{\boldsymbol{x_i}, \boldsymbol{y_i}\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$
- Cost $\boldsymbol{L_i(\theta, x_i, y_i)}$
  - $\boldsymbol{\theta} = \arg\min L_i(\boldsymbol{x_i}, \boldsymbol{y_i})$
- Gradient Descent:
  - Initialize $\boldsymbol{\theta^1}$ with 'random' values (more to that later)
  - $\boldsymbol{\theta^{k+1} = \theta^k - \alpha \nabla_\theta L_i(\theta^k, x_i, y_i)}$

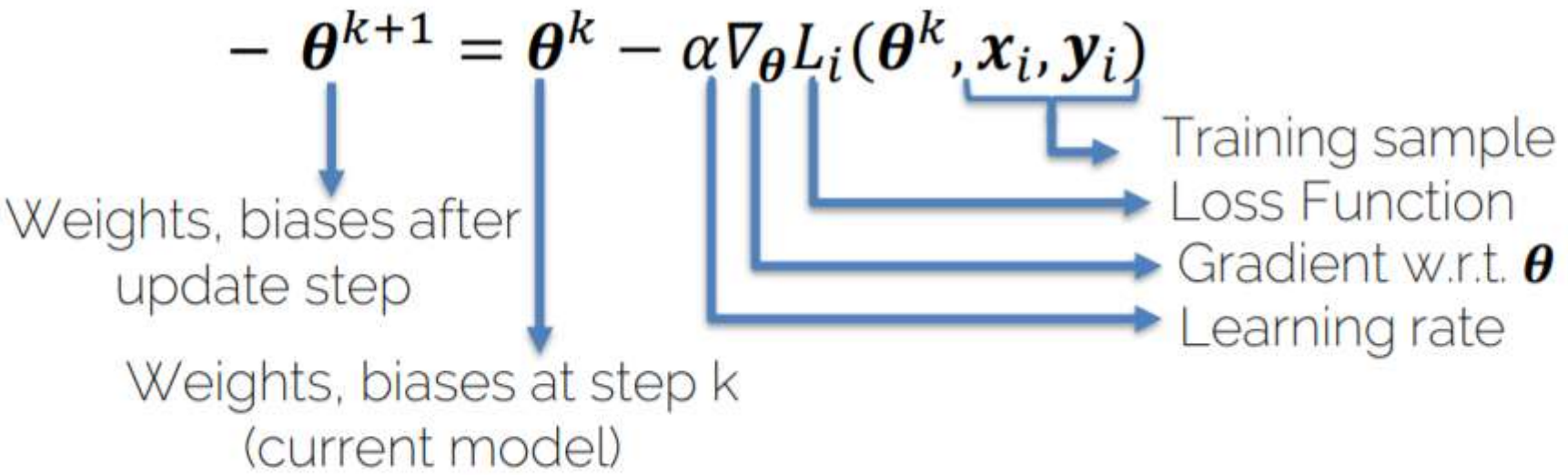
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample

- Given a loss function $\boldsymbol{L}$ and a single training sample $\{\boldsymbol{x_i}, \boldsymbol{y_i}\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$
- Cost $\boldsymbol{L_i(\theta, x_i, y_i)}$
  - $\boldsymbol{\theta} = \arg\min L_i(\boldsymbol{x_i}, \boldsymbol{y_i})$
- Gradient Descent:
  - Initialize $\boldsymbol{\theta^1}$ with 'random' values (more to that later)
  - $\boldsymbol{\theta^{k+1}} = \boldsymbol{\theta^k} - \alpha \nabla_{\boldsymbol{\theta}} \boldsymbol{L_i(\theta^k, x_i, y_i)}$
  - Iterate until convergence: $\left|\boldsymbol{\theta^{k+1}} - \boldsymbol{\theta^k}\right| < \epsilon$

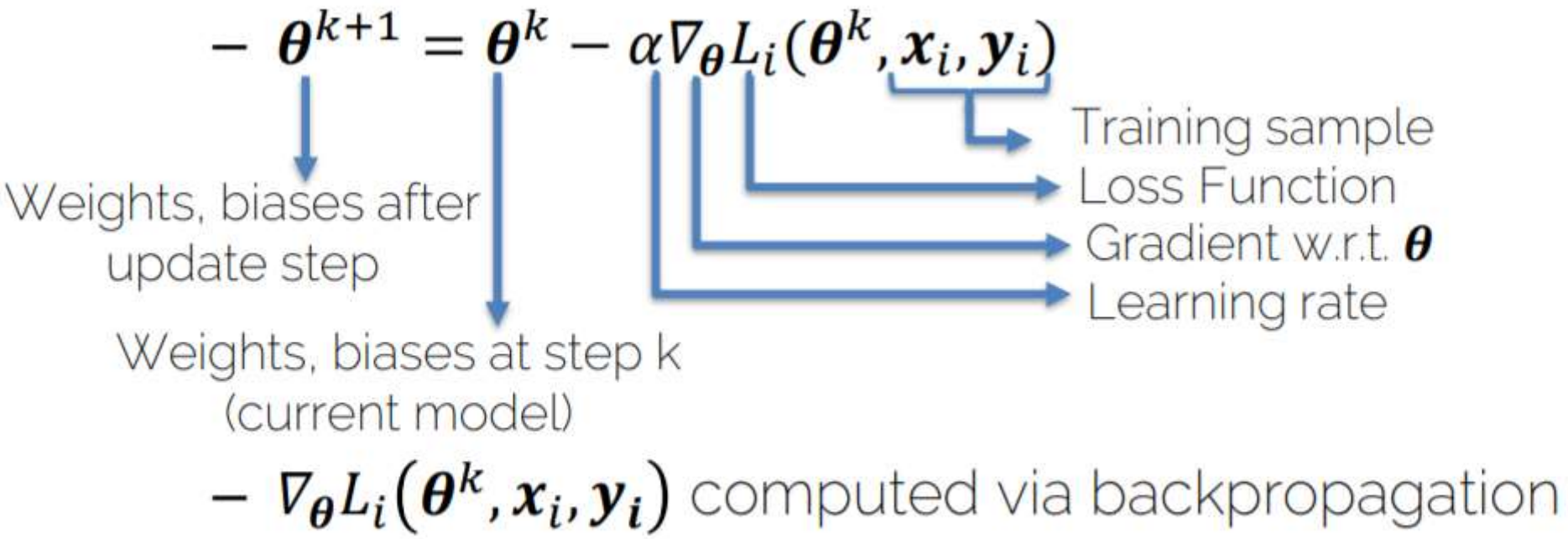
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample



Source: Niessner and Leal-Taixé, Introduction to Deep Learning

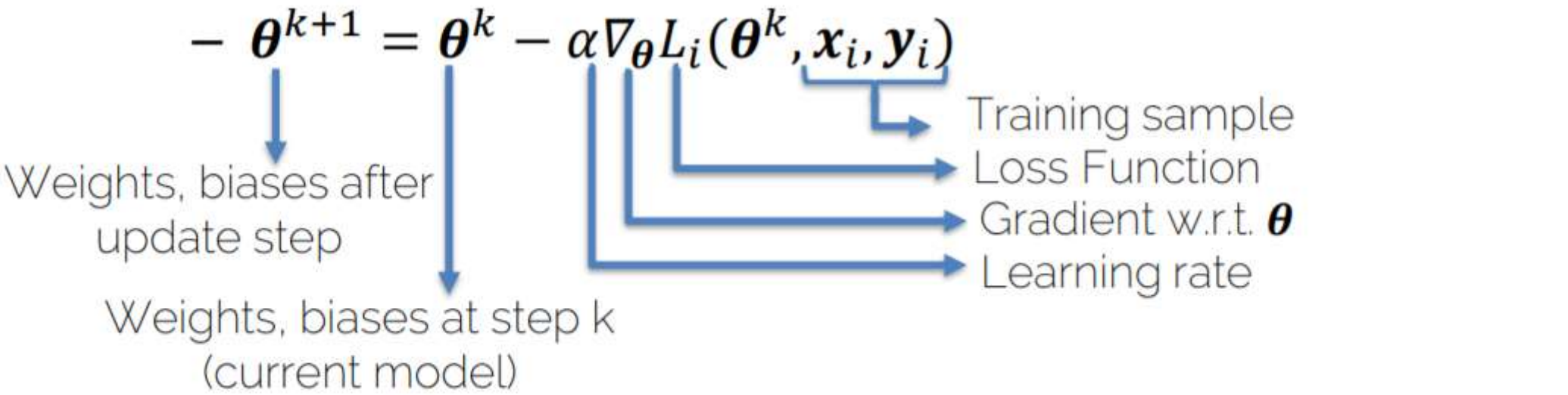
Gradient Descent: Single Training Sample



- $\nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^{k}, \boldsymbol{x}_i, \boldsymbol{y}_i)$ computed via backpropagation

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Single Training Sample



- $\nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^{k}, \boldsymbol{x}_i, \boldsymbol{y}_i)$ computed via backpropagation
- Typically: $\dim\left(\nabla_{\boldsymbol{\theta}} L_i(\boldsymbol{\theta}^{k}, \boldsymbol{x}_i, \boldsymbol{y}_i)\right) = \dim(\boldsymbol{\theta}) \gg 1\ million$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Multiple Training Samples

- Given a loss function L and multiple (n) training samples $\{x_i, y_i\}$

Gradient Descent: Multiple Training Samples

- Given a loss function $\boldsymbol{L}$ and multiple ($\boldsymbol{n}$) training samples $\{\boldsymbol{x_i}, \boldsymbol{y_i}\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Multiple Training Samples

- Given a loss function $\boldsymbol{L}$ and multiple ($\boldsymbol{n}$) training samples $\{\boldsymbol{x_i}, \boldsymbol{y_i}\}$
- Find best model parameters $\boldsymbol{\theta} = \{\boldsymbol{W}, \boldsymbol{b}\}$

- Cost $L = \frac{1}{n}\sum_{i=1}^{n} L_i(\boldsymbol{\theta}, \boldsymbol{x_i}, \boldsymbol{y_i})$
  - $\boldsymbol{\theta} = \arg\min L$

Gradient Descent: Multiple Training Samples

- Update step for multiple samples

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \nabla_{\boldsymbol{\theta}} L\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..n\}}, \boldsymbol{y}_{\{1..n\}}\right)$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Multiple Training Samples

- Update step for multiple samples

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \nabla_{\boldsymbol{\theta}} L\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..n\}}, \boldsymbol{y}_{\{1..n\}}\right)$$

- Gradient is average / sum over residuals

$$\nabla_{\boldsymbol{\theta}} L\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..n\}}, \boldsymbol{y}_{\{1..n\}}\right) = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L_i\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_i, \boldsymbol{y}_i\right)$$

Reminder: this comes from backprop.

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent: Multiple Training Samples

- Update step for multiple samples

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \nabla_{\boldsymbol{\theta}} L\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..n\}}, \boldsymbol{y}_{\{1..n\}}\right)$$

- Gradient is average / sum over residuals

$$\nabla_{\boldsymbol{\theta}} L\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..n\}}, \boldsymbol{y}_{\{1..n\}}\right) = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L_i\left(\boldsymbol{\theta}^{k}, \boldsymbol{x}_i, \boldsymbol{y}_i\right)$$

Reminder: this comes from backprop.

- Often people are lazy and just write: $\nabla L = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L_i$
  - omitting $\frac{1}{n}$ is not 'wrong', it just means rescaling the learning rate

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent on Train Set

- Given large train set with n training samples $\{x_i, y_i\}$
  - Let's say 1 million labeled images
  - Let's say our network has 500k parameters

Gradient Descent on Train Set

- Given large train set with n training samples $\{x_i, y_i\}$
  - Let's say 1 million labeled images
  - Let's say our network has 500k parameters

- Gradient has 500k dimensions
- $n = 1\ million$

→ Extremely expensive to compute

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

The expectation can be approximated with a small subset of the data

$$\mathbb{E}_{i\sim[1,\dots,n]}[L_i(\boldsymbol{\theta}, \boldsymbol{x_i}, \boldsymbol{y_i})] \approx \frac{1}{|S|}\sum_{j\in S}\left(L_j(\boldsymbol{\theta}, \boldsymbol{x_j}, \boldsymbol{y_j})\right) \text{ with } S \subseteq \{1, \dots, n\}$$

Minibatch
choose subset of trainset $\boldsymbol{m \ll n}$

$$B_i = \{\{\boldsymbol{x_1}, \boldsymbol{y_1}\}, \{\boldsymbol{x_2}, \boldsymbol{y_2}\}, \dots, \{\boldsymbol{x_m}, \boldsymbol{y_m}\}\}$$
$$\{B_1, B_2, \dots, B_{n/m}\}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

Minibatch size is hyperparameter

- Typically power of 2 → 8, 16, 32, 64, 128…

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

Minibatch size is hyperparameter

- Typically power of 2 → 8, 16, 32, 64, 128…
- Smaller batch size means greater variance in the gradients

  → noisy updates

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

Minibatch size is hyperparameter

- Typically power of 2 → 8, 16, 32, 64, 128…
- Smaller batch size means greater variance in the gradients

  → noisy updates
- Mostly limited by GPU memory (in backward pass)

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

Minibatch size is hyperparameter

- Typically power of 2 → 8, 16, 32, 64, 128…
- Smaller batch size means greater variance in the gradients

  → noisy updates
- Mostly limited by GPU memory (in backward pass)
- E.g.,
  - Train set has $\mathbf{n} = \mathbf{2^{20}}$ (about 1 million) images

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

Minibatch size is hyperparameter

- Typically power of 2 → 8, 16, 32, 64, 128…
- Smaller batch size means greater variance in the gradients

  → noisy updates
- Mostly limited by GPU memory (in backward pass)
- E.g.,
  - Train set has $\mathbf{n} = 2^{20}$ (about 1 million) images
  - With batch size $\mathbf{m} = 64$: $B_{1 \ldots n/m} = B_{1 \ldots 16{,}384}$ minibatches

(Epoch = complete pass through training set)

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..m\}}, \boldsymbol{y}_{\{1..m\}})$$

k now refers to k-th iteration

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..m\}}, \boldsymbol{y}_{\{1..m\}})$$

$\boldsymbol{k}$ now refers to $\boldsymbol{k}$-th iteration

$$\nabla_{\boldsymbol{\theta}} L = \frac{1}{m} \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} L_i$$

$\boldsymbol{m}$ training samples in the current minibatch

Gradient for the $\boldsymbol{k}$-th minibatch

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Stochastic Gradient Descent (SGD)

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k}, \boldsymbol{x}_{\{1..m\}}, \boldsymbol{y}_{\{1..m\}})$$

$\boldsymbol{k}$ now refers to $\boldsymbol{k}$-th iteration

$$\nabla_{\boldsymbol{\theta}} L = \frac{1}{m} \sum_{i=1}^{m} \nabla_{\boldsymbol{\theta}} L_i$$

$\boldsymbol{m}$ training samples in the current minibatch

Gradient for the $\boldsymbol{k}$-th minibatch

Note the terminology: iteration vs epoch

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems of Stochastic Gradient Descent

- Gradient is scaled equally across all dimensions

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems of Stochastic Gradient Descent

- Gradient is scaled equally across all dimensions
  → i.e., cannot independently scale directions

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems of Stochastic Gradient Descent

- Gradient is scaled equally across all dimensions

  → i.e., cannot independently scale directions

  → need to have conservative min learning rate to avoid divergence

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems of Stochastic Gradient Descent

- Gradient is scaled equally across all dimensions

  → i.e., cannot independently scale directions

  → need to have conservative min learning rate to avoid divergence

  → Slower than 'necessary'

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Problems of Stochastic Gradient Descent

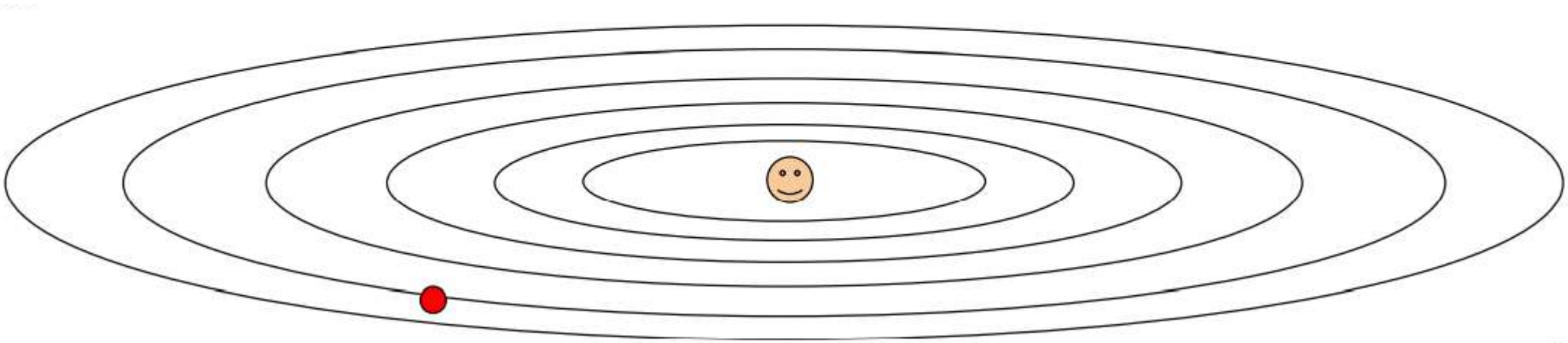
- Gradient is scaled equally across all dimensions
  → i.e., cannot independently scale directions
  → need to have conservative min learning rate to avoid divergence
  → Slower than 'necessary'

- Finding good learning rate is an art by itself

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

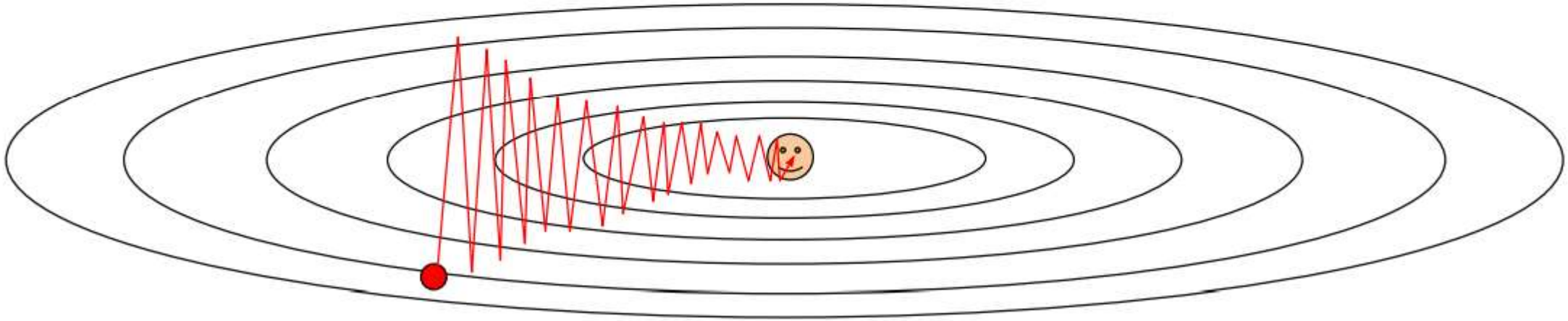
Optimization: Problems with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?



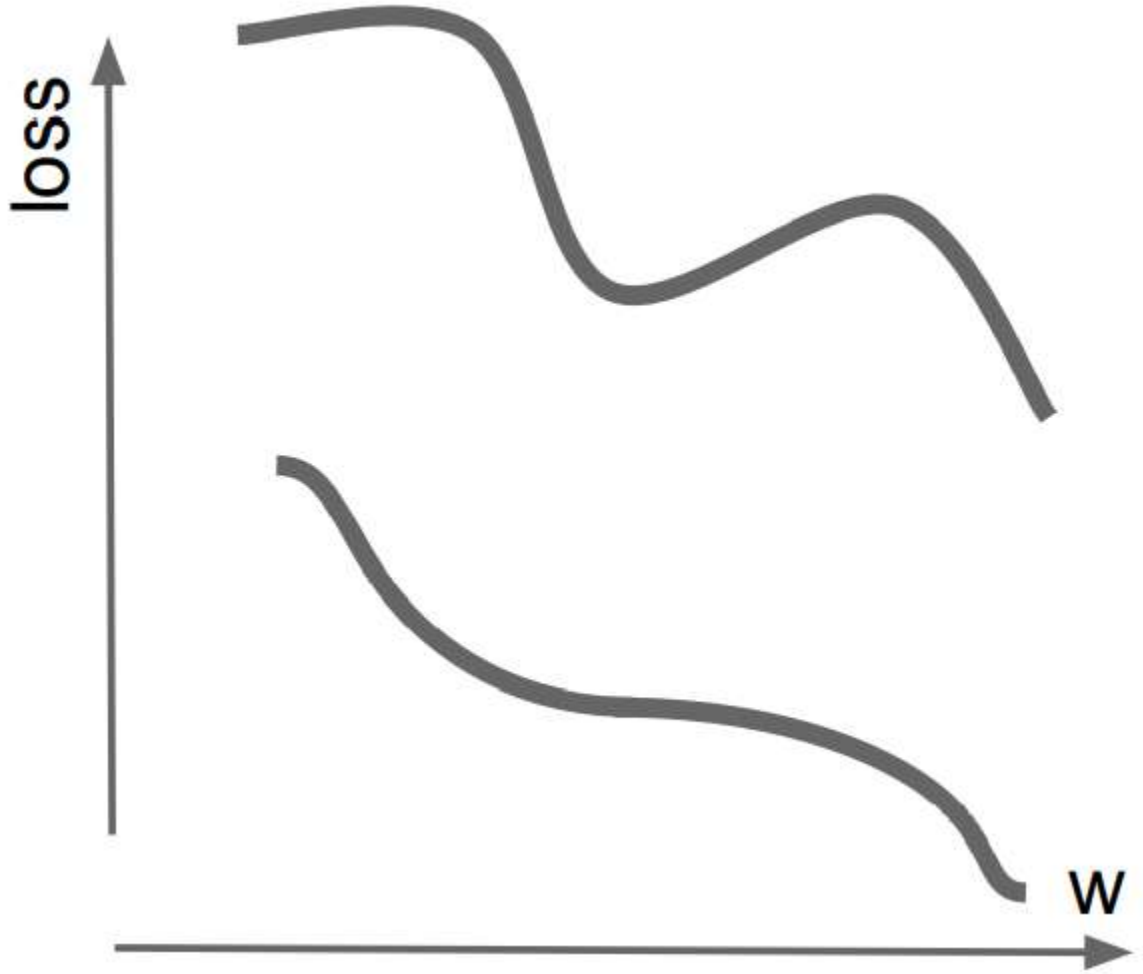
Optimization: Problems with SGD

What if loss changes quickly in one direction and slowly in another?
What does gradient descent do?
Very slow progress along shallow dimension, jitter along steep direction.



Optimization: Problems with SGD

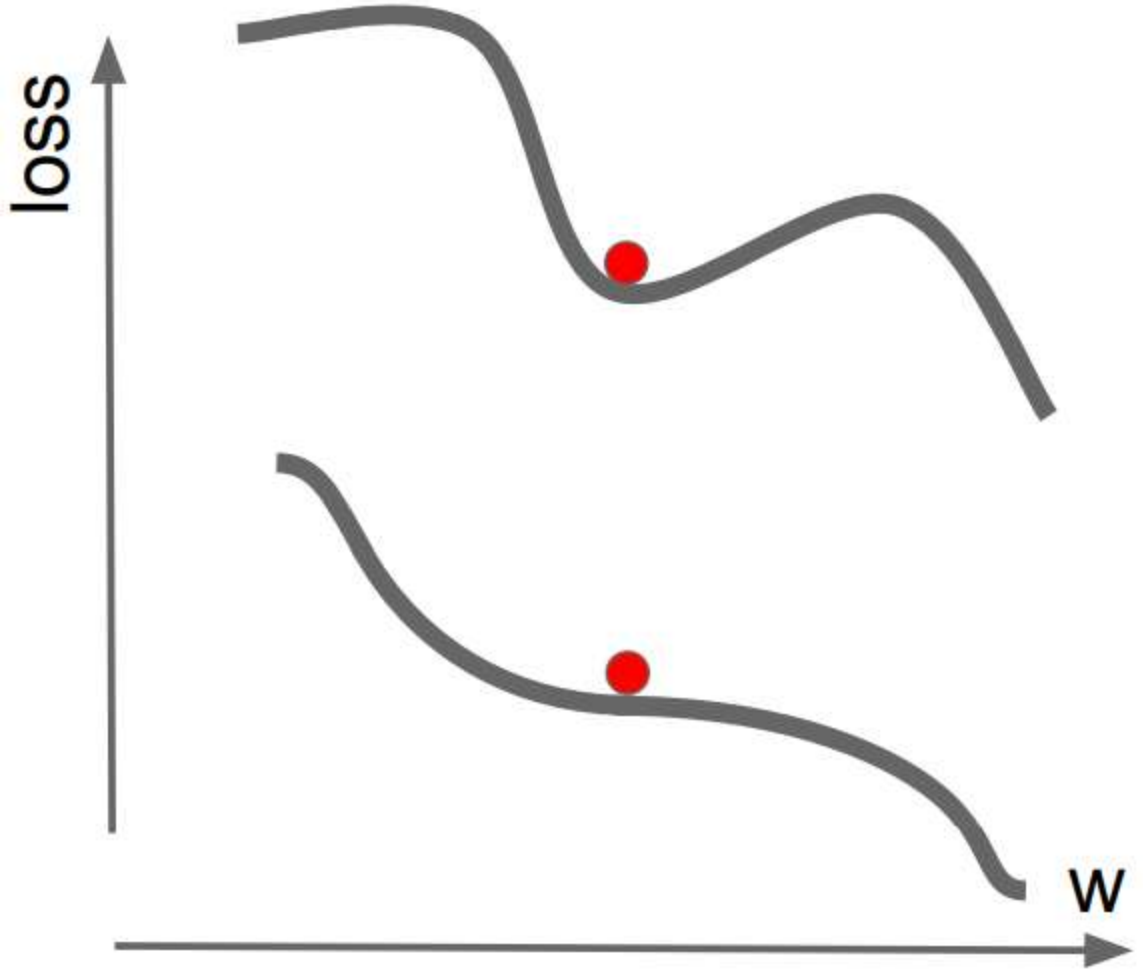
What if the loss function has a local minima or saddle point?



Optimization: Problems with SGD

What if the loss function has a local minima or saddle point?

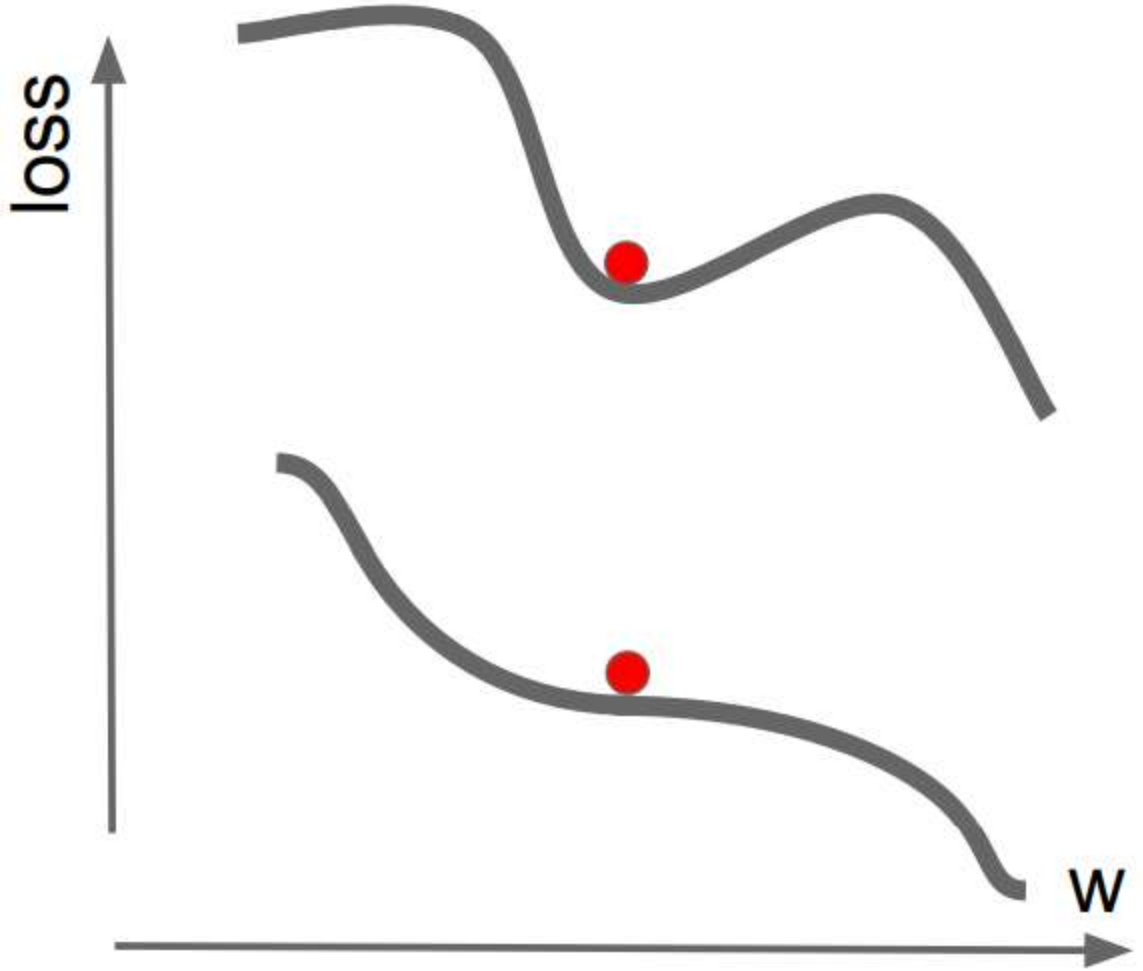
Zero gradient, gradient descent gets stuck



Optimization: Problems with SGD

What if the loss function has a local minima or saddle point?

Saddle points much more common in high dimension



Gradient Descent with Momentum

Gradient Descent with Momentum

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$

accumulation rate ('friction', momentum)

velocity

learning rate

Gradient of current minibatch

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent with Momentum

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$

accumulation rate ('friction', momentum)

velocity

learning rate

Gradient of current minibatch

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

weights of model

velocity

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent with Momentum

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$

accumulation rate ('friction', momentum)

velocity

learning rate

Gradient of current minibatch

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

weights of model

velocity

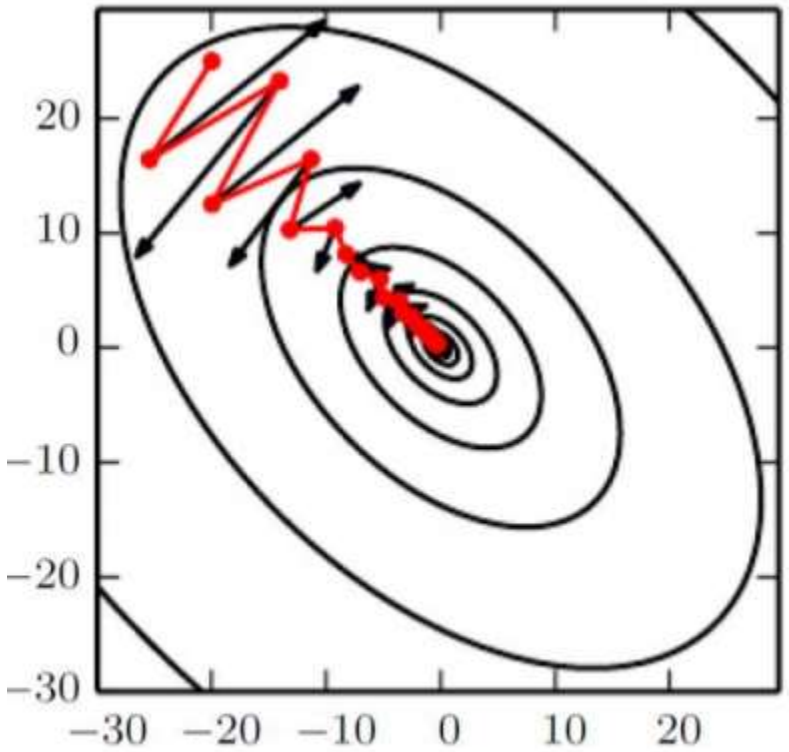
Exponentially-weighted average of gradient

Important: velocity $\boldsymbol{v}^{k}$ is vector-valued!

[Sutskever et al., ICML'13] On the importance of initialization and momentum in deep learning

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent with Momentum



Source: I. Goodfellow

Step will be largest when a sequence of gradients all point to the same direction

Hyperparameters are α, β
β is often set to 0.9

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

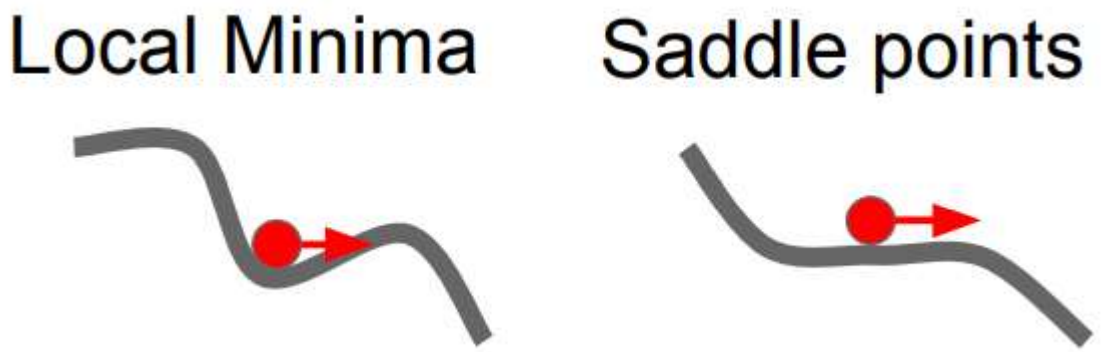
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Gradient Descent with Momentum

- Can it overcome local minima?

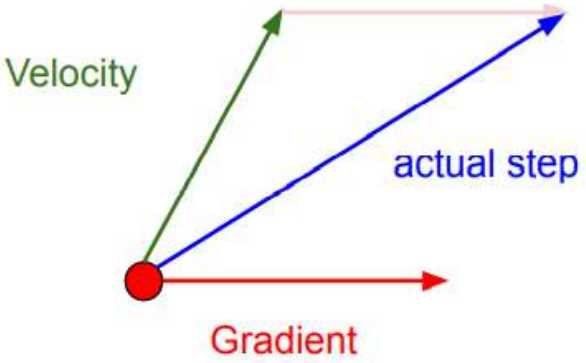
Gradient Descent with Momentum

- Can it overcome local minima?



Nesterov Momentum

Momentum update:



Combine gradient at current point with velocity to get step used to update weights

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$

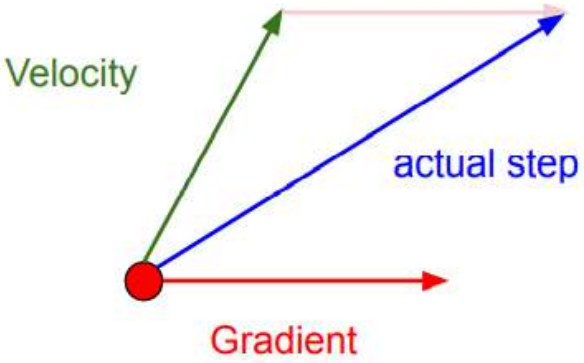
$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning

Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Nesterov Momentum

Momentum update:

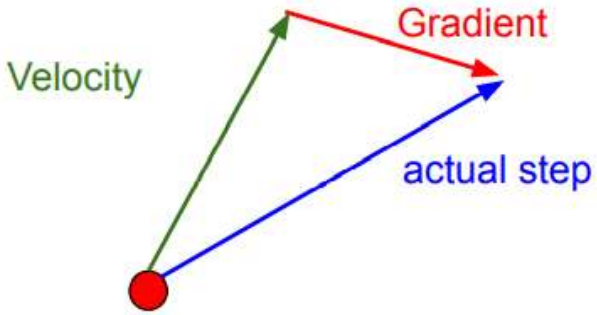


Combine gradient at current point with velocity to get step used to update weights

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

Nesterov Momentum



"Look ahead" to the point where updating using velocity would take us; compute gradient there and mix it with velocity to get actual update direction

$$\tilde{\boldsymbol{\theta}}^{k+1} = \boldsymbol{\theta}^{k} + \beta \cdot \boldsymbol{v}^{k}$$

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\tilde{\boldsymbol{\theta}}^{k+1})$$

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning

Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Nesterov Momentum

- First make a big jump in the direction of the previous accumulated gradient.
- Then measure the gradient where you end up and make a correction.

standard momentum

$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$
$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

brown vector = jump, red vector = correction, green vector = accumulated gradient

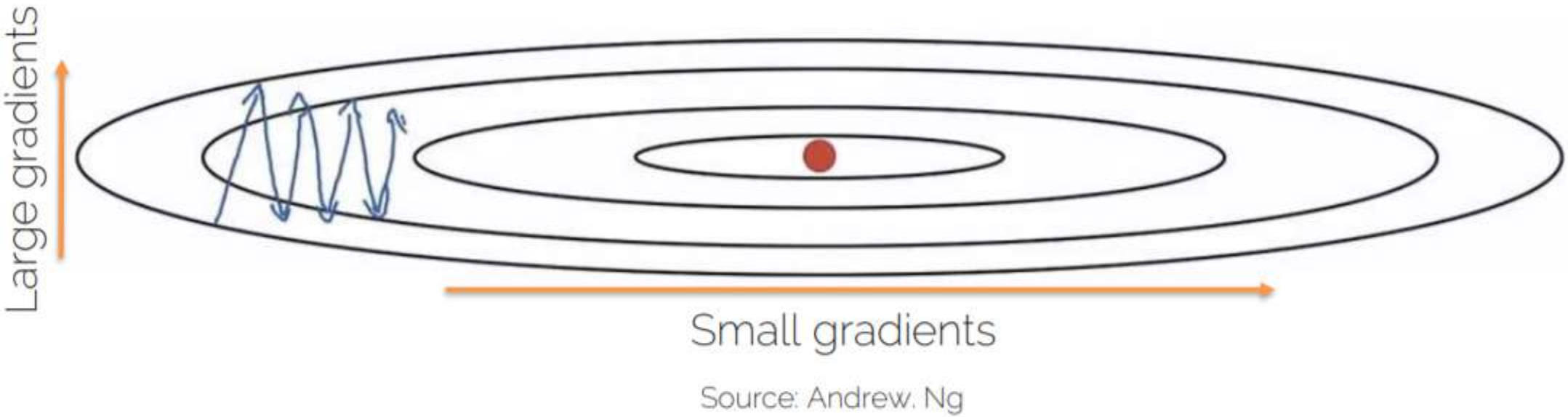
blue vectors = standard momentum

Source: G. Hinton

$$\tilde{\boldsymbol{\theta}}^{k+1} = \boldsymbol{\theta}^{k} + \beta \cdot \boldsymbol{v}^{k}$$
$$\boldsymbol{v}^{k+1} = \beta \cdot \boldsymbol{v}^{k} - \alpha \cdot \nabla_{\boldsymbol{\theta}} L(\tilde{\boldsymbol{\theta}}^{k+1})$$
$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} + \boldsymbol{v}^{k+1}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Root Mean Squared Prop (RMSProp)



Source: Andrew. Ng

- RMSProp divides the learning rate by an exponentially-decaying average of squared gradients.

Hinton et al. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSERA: Neural networks for machine learning 4.2 (2012): 26-31.

RMSProp

$$\boldsymbol{s}^{k+1} = \beta \cdot \boldsymbol{s}^{k} + (1-\beta)[\nabla_{\boldsymbol{\theta}} L \circ \nabla_{\boldsymbol{\theta}} L]$$

Element-wise multiplication

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \cdot \frac{\nabla_{\boldsymbol{\theta}} L}{\sqrt{\boldsymbol{s}^{k+1}} + \epsilon}$$

Hyperparameters: α, β, ϵ

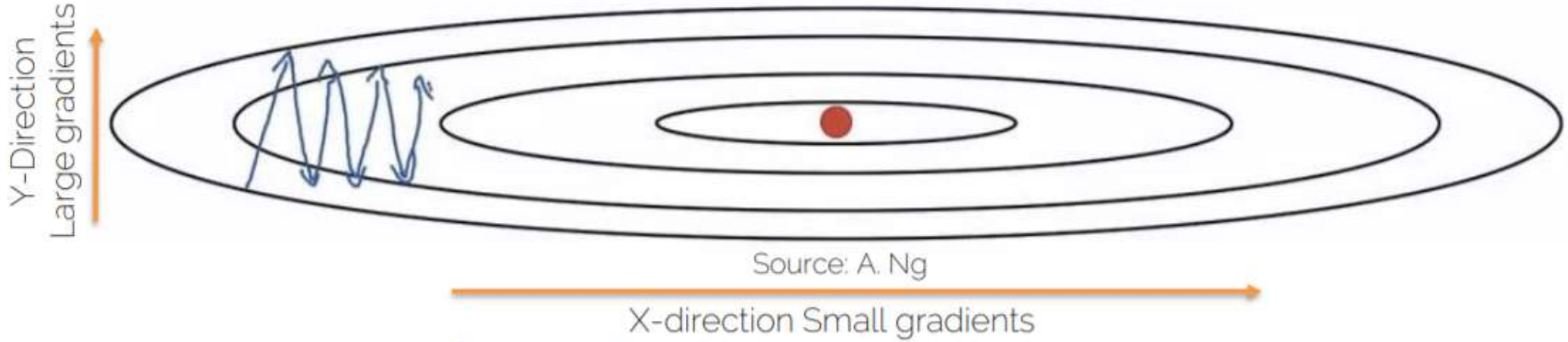
Needs tuning!

Often 0.9

Typically 10^{-8}

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

RMSProp



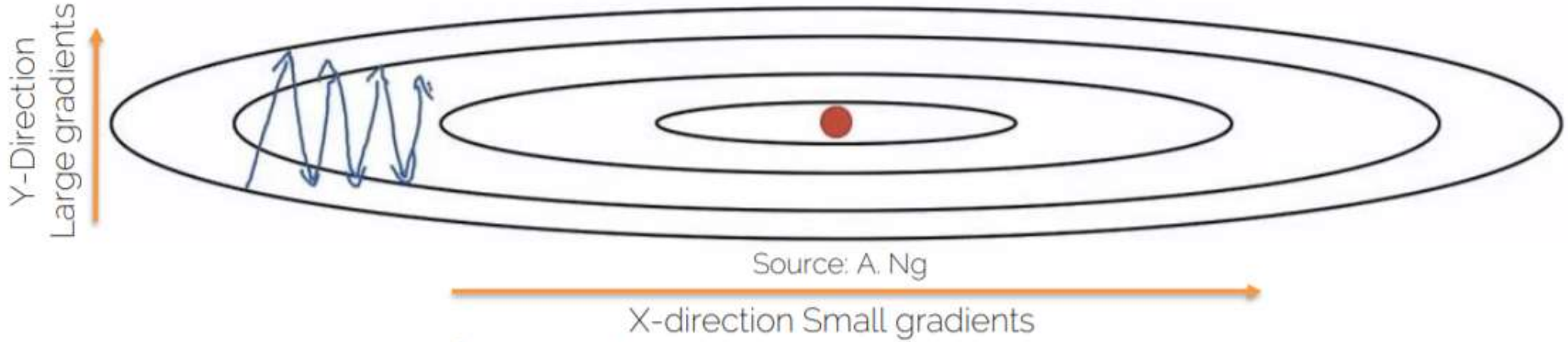
(Uncentered) variance of gradients
→ second momentum

$$\boldsymbol{s}^{k+1} = \beta \cdot \boldsymbol{s}^{k} + (1-\beta)[\nabla_{\boldsymbol{\theta}} L \circ \nabla_{\boldsymbol{\theta}} L]$$

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \cdot \frac{\nabla_{\boldsymbol{\theta}} L}{\sqrt{\boldsymbol{s}^{k+1}} + \epsilon}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

RMSProp



(Uncentered) variance of gradients
→ second momentum

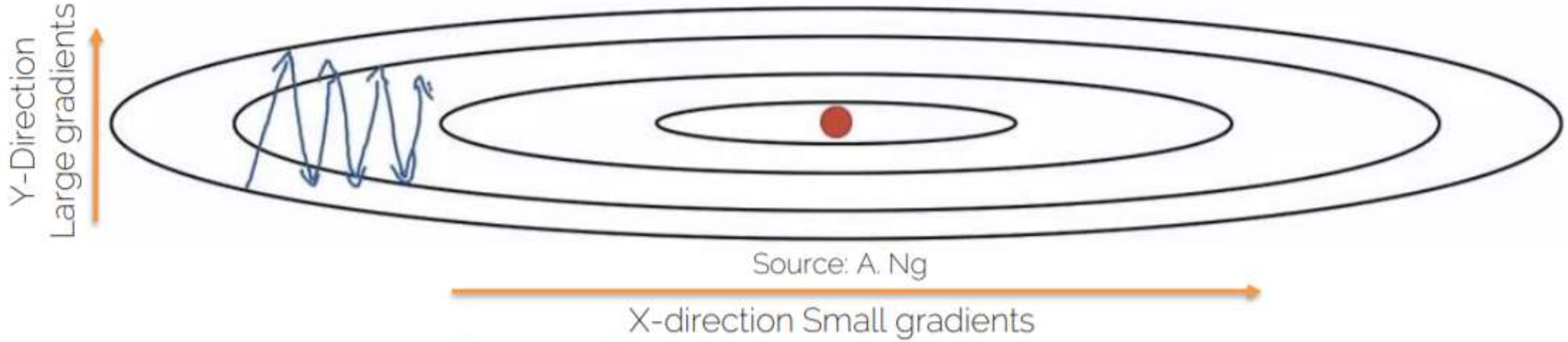
$$\boldsymbol{s}^{k+1} = \beta \cdot \boldsymbol{s}^{k} + (1-\beta)[\nabla_{\boldsymbol{\theta}} L \circ \nabla_{\boldsymbol{\theta}} L]$$

We're dividing by square gradients:
- Division in Y-Direction will be large
- Division in X-Direction will be small

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \cdot \frac{\nabla_{\boldsymbol{\theta}} L}{\sqrt{\boldsymbol{s}^{k+1}} + \epsilon}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

RMSProp



(Uncentered) variance of gradients
→ second momentum

$$\boldsymbol{s}^{k+1} = \beta \cdot \boldsymbol{s}^{k} + (1-\beta)[\nabla_{\boldsymbol{\theta}} L \circ \nabla_{\boldsymbol{\theta}} L]$$

We're dividing by square gradients:
- Division in Y-Direction will be large
- Division in X-Direction will be small

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^{k} - \alpha \cdot \frac{\nabla_{\boldsymbol{\theta}} L}{\sqrt{\boldsymbol{s}^{k+1}} + \epsilon}$$

Can increase learning rate!

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

RMSProp

- Dampening the oscillations for high-variance directions

- Can use faster learning rate because it is less likely to diverge
  - Speed up learning speed
  - Second moment

Adaptive Moment Estimation (Adam)

Adaptive Moment Estimation (Adam)

Idea : Combine Momentum and RMSProp

Adaptive Moment Estimation (Adam)

Idea : Combine Momentum and RMSProp

$$\boldsymbol{m}^{k+1} = \beta_1 \cdot \boldsymbol{m}^{k} + (1-\beta_1)\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})$$

First momentum: mean of gradients

$$\boldsymbol{v}^{k+1} = \beta_2 \cdot \boldsymbol{v}^{k} + (1-\beta_2)[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k}) \circ \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k})]$$

Second momentum: variance of gradients

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Adaptive Moment Estimation (Adam)

Idea : Combine Momentum and RMSProp

$$\boldsymbol{m}^{k+1} = \beta_1 \cdot \boldsymbol{m}^k + (1-\beta_1)\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k)$$

First momentum: mean of gradients

$$\boldsymbol{v}^{k+1} = \beta_2 \cdot \boldsymbol{v}^k + (1-\beta_2)[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k) \circ \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k)]$$

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \cdot \frac{\boldsymbol{m}^{k+1}}{\sqrt{\boldsymbol{v}^{k+1}} + \epsilon}$$

Note : This is not the update rule of Adam

Second momentum: variance of gradients

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Adaptive Moment Estimation (Adam)

Idea : Combine Momentum and RMSProp

$$\boldsymbol{m}^{k+1} = \beta_1 \cdot \boldsymbol{m}^k + (1-\beta_1)\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k)$$

First momentum: mean of gradients

$$\boldsymbol{v}^{k+1} = \beta_2 \cdot \boldsymbol{v}^k + (1-\beta_2)[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k) \circ \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k)]$$

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \cdot \frac{\boldsymbol{m}^{k+1}}{\sqrt{\boldsymbol{v}^{k+1}} + \epsilon}$$

Note : This is not the update rule of Adam

Second momentum: variance of gradients

Q. What happens at $\boldsymbol{k} = 0$?
A. We need bias correction as $\boldsymbol{m}^0 = 0$ and $\boldsymbol{v}^0 = 0$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Adam : Bias Corrected

- Combines Momentum and RMSProp

$$\boldsymbol{m}^{k+1} = \beta_1 \cdot \boldsymbol{m}^k + (1-\beta_1)\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k) \qquad \boldsymbol{v}^{k+1} = \beta_2 \cdot \boldsymbol{v}^k + (1-\beta_2)[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k) \circ \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k)$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Adam : Bias Corrected

- Combines Momentum and RMSProp

$$\boldsymbol{m}^{k+1} = \beta_1 \cdot \boldsymbol{m}^k + (1-\beta_1)\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k) \qquad \boldsymbol{v}^{k+1} = \beta_2 \cdot \boldsymbol{v}^k + (1-\beta_2)[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k) \circ \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^k)$$

- $\boldsymbol{m}^k$ and $\boldsymbol{v}^k$ are initialized with zero
  - → bias towards zero
  - → Need bias-corrected moment updates

Update rule of Adam

$$\hat{\boldsymbol{m}}^{k+1} = \frac{\boldsymbol{m}^{k+1}}{1-\beta_1{}^{k+1}} \qquad \hat{\boldsymbol{v}}^{k+1} = \frac{\boldsymbol{v}^{k+1}}{1-\beta_2{}^{k+1}} \longrightarrow \boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \alpha \cdot \frac{\hat{\boldsymbol{m}}^{k+1}}{\sqrt{\hat{\boldsymbol{v}}^{k+1}}+\epsilon}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

Adam

- Exponentially-decaying mean and variance of gradients (combines first and second order momentum)

- Hyperparameters: α, β_1, β_2, ϵ

$$m^{k+1} = \beta_1 \cdot m^k + (1-\beta_1)\nabla_\theta L(\theta^k)$$

$$v^{k+1} = \beta_2 \cdot v^k + (1-\beta_2)\left[\nabla_\theta L(\theta^k) \circ \nabla_\theta L(\theta^k)\right]$$

$$\hat{m}^{k+1} = \frac{m^{k+1}}{1-\beta_1^{k+1}} \qquad \hat{v}^{k+1} = \frac{v^{k+1}}{1-\beta_2^{k+1}}$$

$$\theta^{k+1} = \theta^k - \alpha \cdot \frac{\hat{m}^{k+1}}{\sqrt{\hat{v}^{k+1}} + \epsilon}$$

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

There are a few others…

- ‘Vanilla’ SGD
- Momentum
- RMSProp
- Adagrad
- Adadelta
- AdaMax
- Nada
- AMSGrad

Adam is mostly method of choice for neural networks!

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

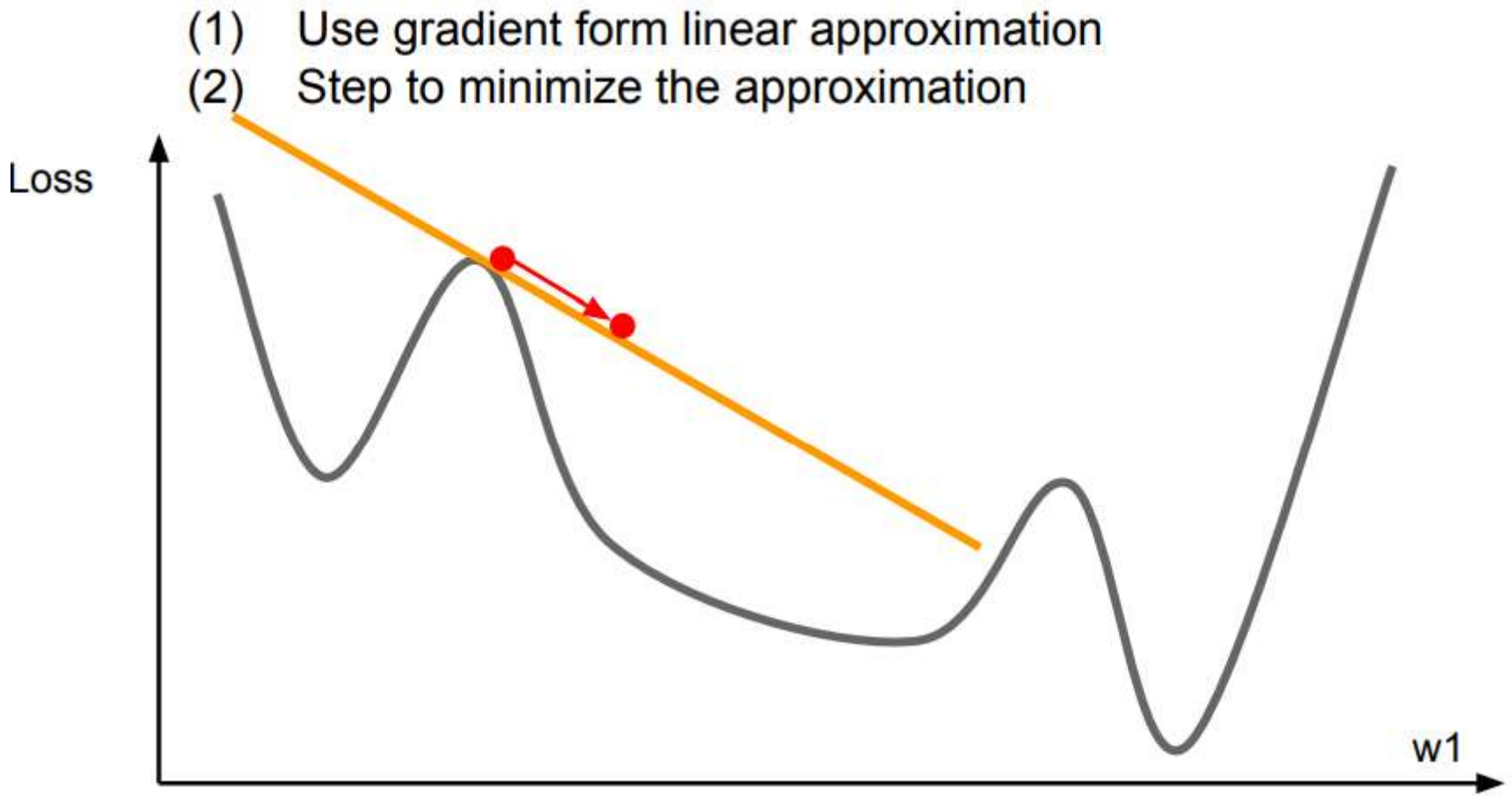
Jacobian and Hessian

- Derivative $\boldsymbol{f}: \mathbb{R} \rightarrow \mathbb{R}$ $\frac{df(x)}{dx}$
- Gradient $\boldsymbol{f}: \mathbb{R}^m \rightarrow \mathbb{R}$ $\nabla_{\boldsymbol{x}} f(\boldsymbol{x})$ $\left(\frac{\mathrm{d}f(\boldsymbol{x})}{\mathrm{d}x_1}, \frac{\mathrm{d}f(\boldsymbol{x})}{\mathrm{d}x_2}\right)$
- Jacobian $\boldsymbol{f}: \mathbb{R}^m \rightarrow \mathbb{R}^n$ $\mathbf{J} \in \mathbb{R}^{n \times m}$
- Hessian $\boldsymbol{f}: \mathbb{R}^m \rightarrow \mathbb{R}$ $\mathbf{H} \in \mathbb{R}^{m \times m}$

SECOND DERIVATIVE

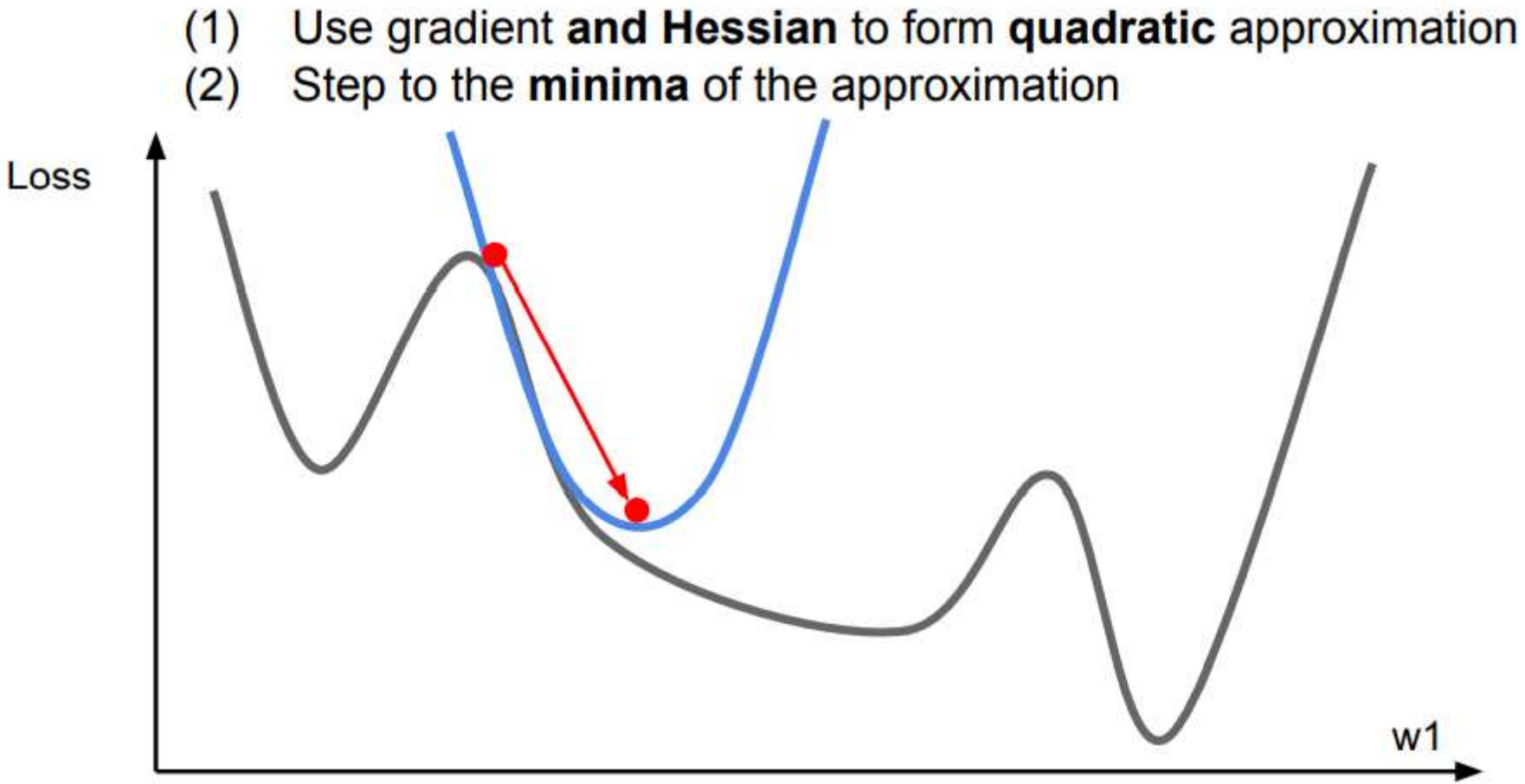
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

First-Order Optimization



Source: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Second-Order Optimization



Source: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Second-Order Optimization

second-order Taylor expansion:

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \boldsymbol{H} (\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning

Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Second-Order Optimization

second-order Taylor expansion:

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Q: Why is this bad for deep learning?

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning

Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Second-Order Optimization

second-order Taylor expansion:

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Hessian has O(N^2) elements

Q: Why is this bad for deep learning?

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning

Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Second-Order Optimization

second-order Taylor expansion:

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Q: Why is this bad for deep learning?

Hessian has O(N^2) elements
Inverting takes O(N^3)

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning
Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Second-Order Optimization

second-order Taylor expansion:

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^\top \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Q: Why is this bad for deep learning?

Hessian has O(N^2) elements
Inverting takes O(N^3)
N = (Tens or Hundreds of) Millions

Adapted from: Niessner and Leal-Taixé, Introduction to Deep Learning
Adapted from: Fei-Fei Li, Ranjay Krishna, Danfei Xu, CS231n: Convolutional Neural Networks for Visual Recognition

Which, What, and When?

- Standard: **Adam**
- Fallback option: **SGD with momentum**
- **Newton, L-BFGS, GN, LM** only if you can do full batch updates (doesn't work well for minibatches!!)

This practically never happens for DL
Theoretically, it would be nice though due to fast convergence

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

General Optimization

- Linear Systems (Ax = b)
  - LU, QR, Cholesky, Jacobi, Gauss-Seidel, CG, PCG, etc.
- Non-linear (gradient-based)
  - Newton, Gauss-Newton, LM, (L)BFGS ← second order
  - Gradient Descent, SGD ← first order
- Others
  - Genetic algorithms, MCMC, Metropolis-Hastings, etc.
  - Constrained and convex solvers (Langrage, ADMM, Primal-Dual, etc.)

Source: Niessner and Leal-Taixé, Introduction to Deep Learning